EAST WEST UNIVERSITY

Department of Computer Science and Engineering (CSE)

Project Report

Title: CPU Scheduling Algorithm Simulator and Evaluator

Course Title: Operating Systems

Course Code: CSE325

Section: 11

Submitted By:

Student Name: Kamrul Hasan

Student ID: 2023-3-60-148

Submitted to:

Instructor Name: Dr. Md. Motaharul Islam, Adjunct Professor

Department of Computer Science and Engineering

Date of Performance: 29-03-2026

CPU Scheduling Algorithm Simulator and Evaluator

1. Introduction

CPU scheduling is a key concept in operating systems. It decides which process will be assigned to the CPU when multiple processes are waiting in the ready queue. Proper scheduling helps improve overall system performance by reducing waiting time and increasing CPU efficiency.

Various scheduling algorithms are used to control how processes are executed. Each algorithm follows a different approach to select the next process for execution. In this project, a CPU Scheduling Algorithm Simulator has been created using the C programming language. The simulator demonstrates different scheduling techniques and computes the waiting time as well as the average waiting time for each process.

2. Objectives of the Project

The primary goals of this project are:

- To gain a clear understanding of CPU scheduling concepts.
- To implement widely used CPU scheduling algorithms in C.
- To simulate the execution of processes in a CPU environment.
- To compute individual waiting times and the average waiting time.
- To generate a Gantt chart showing the order of process execution.

3. Algorithms Implemented

(a) First Come First Served (FCFS)

First Come First Served (FCFS) is one of the most basic CPU scheduling algorithms. In this approach, processes are executed in the exact order in which they arrive in the ready queue. The process that arrives earliest gets access to the CPU first.

While FCFS is easy to understand and implement, it can sometimes result in increased waiting times, especially when shorter processes are delayed behind longer ones.

(b) Shortest Job First (SJF)

Shortest Job First scheduling selects the process with the smallest burst time to

execute next. This algorithm helps reduce the average waiting time compared to

FCFS. However, it requires knowledge of the burst time in advance.

(c) Round Robin (RR)

Round Robin scheduling assigns a fixed time slice, known as a time quantum, to each process. Every process is allowed to use the CPU for this limited duration. If a process does not complete within its allocated time, it is moved back to the end of the ready queue to wait for its next turn.

This approach ensures that all processes get a fair share of CPU time, making it especially suitable for time-sharing systems.

(d) Priority Scheduling

In Priority Scheduling, each process is given a priority level. The CPU is allocated to the process with the highest priority first. If multiple processes have the same priority, they are executed based on their arrival order.

Although this method ensures important processes are handled first, it may cause lower-priority processes to wait for a long time.

4. Program Design and Implementation

The program is developed using the C programming language. A structure is defined to store essential information for each process, such as process ID, burst time, and priority.

The application follows a menu-driven design. Initially, the user provides the number of processes along with their burst times and priorities. After that, the user selects a preferred scheduling algorithm from the menu.

Depending on the chosen algorithm, the program performs the following tasks:

- Calculates the waiting time for each process
- Computes the average waiting time
- Displays a Gantt chart to illustrate the execution sequence

Each scheduling algorithm is implemented in a separate function, making the program modular, organized, and easier to maintain.

To improve reliability, the program includes input validation to handle incorrect entries, such as zero or negative burst times.

Additionally, a comparison feature is included to identify the most efficient scheduling algorithm based on the lowest average waiting time. The Shortest Job First (SJF) algorithm is implemented in both preemptive and non-preemptive versions for better analysis.

Implementation of CPU Scheduling Algorithms

1. Main Program Structure

```
int main() {
    int n, bt[MAX], at[MAX], choice, quantum, priority[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Times:\n");
    for (int i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &at[i]);
    }
    printf("Enter Burst Times:\n");
    for (int i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    while (1) {
        printf("\n===== CPU Scheduling Simulator =====\n");
        printf("1. FCFS\n2. SJF\n3. Round Robin\n");
        printf("4. Priority\n5. SRTF\n6. Preemptive Priority\n7. Exit\n");
        scanf("%d", &choice);
        switch (choice) {
            case 1: fcfs(n, bt); break;
            case 2: sjf(n, bt); break;
            case 3:
                printf("Enter Quantum: ");
                scanf("%d", &quantum);
```

```
                roundRobin(n, bt, quantum);
                break;
            case 4:
                for (int i = 0; i < n; i++)
                    scanf("%d", &priority[i]);
                priorityScheduling(n, bt, priority);
                break;
            case 5: srtf(n, at, bt); break;
            case 6:
                for (int i = 0; i < n; i++)
                    scanf("%d", &priority[i]);
                priorityPreemptive(n, at, bt, priority);
                break;
            case 7: return 0;
        }
    }
}
```

2. Gantt Chart Representation

```
printf("\nGantt Chart:\n|");
printf(" P%d |", i+1);
```

3. Preemptive SJF (SRTF) Algorithm Logic

```
if (at[i] <= time && rt[i] < min && rt[i] > 0) {
    min = rt[i];
    shortest = i;
}
```

4. Preemptive Priority Scheduling Logic

```
if (at[i] <= time && rt[i] > 0 && pr[i] < highest) {
   highest = pr[i];
   idx = i;
}
```

5. WAITING TIME CALCULATION

```
wt[i] = finish_time - bt[i] - at[i];
```

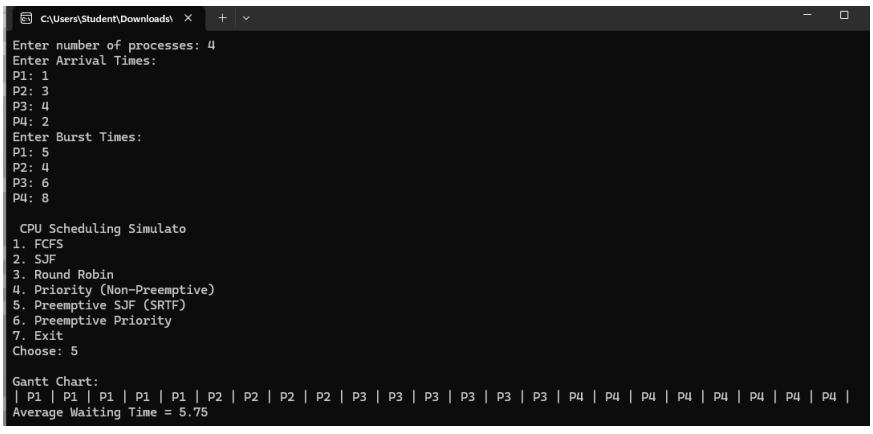
6. Round Robin Scheduling Mechanism

```
if (rem[i] > quantum) {
   time += quantum;
   rem[i] -= quantum;
}
```

Sample Output

The program asks the user to input process information such as burst time and priority. After selecting an algorithm from the menu, the program displays the Gantt chart, waiting time for each process, and the average waiting time.

- **Preemptive SJF:**



- **SJF:**

```
 CPU Scheduling Simulato
1. FCFS
2. SJF
3. Round Robin
4. Priority (Non-Preemptive)
5. Preemptive SJF (SRTF)
6. Preemptive Priority
7. Exit
Choose: 2

Gantt Chart:
| P2 | P1 | P3 | P4 |
Average Waiting Time = 7.00
```

- **Round Robin:**

```
 CPU Scheduling Simulation
1. FCFS
2. SJF
3. Round Robin
4. Priority (Non-Preemptive)
5. Preemptive SJF (SRTF)
6. Preemptive Priority
7. Exit
Choose: 3
Enter Time Quantum: 4

Gantt Chart:
| P1 | P2 | P3 | P2 | P3 |
Average Waiting Time = 6.33
```

Conclusion

This project demonstrates the fundamental working principles of various CPU scheduling algorithms. By implementing FCFS, SJF, Round Robin, Priority Scheduling, as well as their preemptive versions, the program illustrates how different algorithms influence process waiting times and CPU allocation. The simulator enhances understanding of the behavior and efficiency of scheduling techniques used in operating systems. It provides practical insight into CPU

scheduling mechanisms and helps develop problem-solving and programming skills using the C language.